VN-ALPR

Hệ thống nhận diện biển số xe Việt Nam
cho cổng bãi đỗ xe

Báo cáo dự án Deep Learning

Đoàn Trí Hùng

Mã nguồn: `https://github.com/hungdt31/vn-alpr`

Tháng 10 năm 2026

Tóm tắt

Bối cảnh. Báo cáo trình bày VN-ALPR, một hệ thống nhận diện biển số xe Việt Nam từ đầu đến cuối (end-to-end), phục vụ việc ghi nhận xe vào/ra tại cổng bãi đỗ xe. Hệ thống phải đọc được cả biển 1 dòng của ô tô lẫn biển 2 dòng của xe máy, và chạy được thời gian thực trên video.

Phương pháp. Pipeline gồm năm khối: bộ phát hiện YOLO11n được fine-tune; khối tiền xử lý cắt, chỉnh nghiêng và tách biển 2 dòng; bộ nhận dạng CRNN (CNN + BiLSTM + CTC) huấn luyện trên biển số tổng hợp; khối hậu xử lý theo mẫu định dạng biển Việt Nam; và ByteTrack kết hợp bỏ phiếu qua nhiều khung hình. Mô hình được đưa vào một bãi giữ xe mô phỏng 3D (React, FastAPI, SQLite): người gửi lấy thẻ sau khi biển số được nhận diện và có thể sửa tay, còn khi lấy xe, biển đọc ở cổng ra phải khớp với thẻ.

Kết quả. Bộ phát hiện đạt mAP@0.5 = 0,995 và mAP@0.5:0.95 = 0,713 trên tập test, vượt cả hai chỉ tiêu. Hệ thống chạy khoảng 13 FPS trên CPU laptop. CRNN đọc đúng 95,7 % biển tổng hợp chưa từng thấy nhưng 0 % biển thật trong bộ dữ liệu.

Kết luận. Phân tích lỗi chỉ ra ba nguyên nhân cộng dồn: khoảng cách miền giữa dữ liệu tổng hợp và biển thật, việc phân loại 1/2 dòng bằng tỉ lệ khung thất bại trên ảnh bị co giãn (71 % biển 1 dòng bị cắt đôi), và nhãn chữ của tập test chưa được duyệt thật. Thứ tự cải thiện ưu tiên là sửa cách phân loại số dòng, duyệt nhãn, rồi fine-tune CRNN trên biển thật.

Từ khoá: nhận diện biển số xe, YOLO, CRNN, CTC, ByteTrack, OCR, dữ liệu tổng hợp, biển số Việt Nam.

Mục lục

1 Giới thiệu **4**
1.1 Bài toán . . . 4
1.2 Mục tiêu . . . 4
1.3 Đóng góp . . . 4

2 Cơ sở lý thuyết và công trình liên quan **5**

3 Dữ liệu **5**
3.1 Dữ liệu phát hiện biển số . . . 5
3.2 Dữ liệu nhận dạng chữ tổng hợp . . . 6
3.3 Nhãn chữ thật: gán nhãn bán tự động . . . 6

4 Phương pháp **7**
4.1 Kiến trúc tổng thể . . . 7
4.2 Phát hiện biển số . . . 7
4.3 Tiền xử lý . . . 7
4.4 Nhận dạng ký tự: CRNN . . . 8
4.5 Hậu xử lý theo định dạng biển số Việt Nam . . . 9
4.6 Theo dõi và bỏ phiếu qua nhiều khung hình . . . 9
4.7 Nhật ký cổng . . . 10

5 Ứng dụng: hệ thống bãi giữ xe mô phỏng **10**
5.1 Luồng nghiệp vụ . . . 10
5.2 Luật nghiệp vụ . . . 12
5.3 Giao diện . . . 13
5.4 Kiến trúc backend . . . 13

6 Thiết lập thực nghiệm **14**
6.1 Môi trường . . . 14
6.2 Siêu tham số . . . 14
6.3 Độ đo . . . 14

7 Kết quả **15**
7.1 Bộ phát hiện . . . 15
7.2 Bộ nhận dạng . . . 15
7.3 Tốc độ . . . 16

8 Phân tích lỗi **16**

8.1 Khoảng cách miền của bộ nhận dạng 17

8.2 Phân loại 1 dòng / 2 dòng bằng tỉ lệ khung 17

8.3 Chất lượng nhãn chữ 18

8.4 Nhận xét chung 18

9 Hạn chế và hướng phát triển **18**

10 Kết luận **19**

A Cấu trúc mã nguồn **20**

B Tái lập kết quả **20**

1 Giới thiệu

1.1 Bài toán

Tại cổng ra vào bãi đỗ xe của chung cư, trung tâm thương mại hay nhà máy, việc ghi biển số thường làm thủ công hoặc dùng thẻ giấy. Cách làm này chậm, dễ sai và khó tra cứu. Bài toán nhận diện biển số tự động (Automatic License Plate Recognition, ALPR) nhận ảnh hoặc video từ camera và trả về chuỗi ký tự của biển số. Kết quả đó được dùng để ghi nhận lượt vào/ra và tính thời gian gửi xe.

Biển số Việt Nam có hai đặc thù làm bài toán khó hơn so với nhiều nước khác:

- **Có cả biển 1 dòng và 2 dòng.** Ô tô thường dùng biển 1 dòng (ví dụ `51G-123.45`). Xe máy và một số ô tô dùng biển 2 dòng (ví dụ `59-X1 / 123.45`).
- **Định dạng có cấu trúc nhưng đa dạng.** Seri có thể gồm 1–2 chữ cái, hoặc chữ cái kèm chữ số với xe máy, và phần số có 4–5 chữ số.

1.2 Mục tiêu

Các chỉ tiêu được đặt ra trong đặc tả dự án như sau:

Bảng 1: Chỉ tiêu mục tiêu của dự án.

Chỉ số	Mục tiêu	Ghi chú
mAP@0.5 của bộ phát hiện	$\geq 0{,}95$	Trên tập test
mAP@0.5:0.95 của bộ phát hiện	$\geq 0{,}70$	
Độ chính xác ký tự	$\geq 97\,\%$	
Độ chính xác biển (khớp hoàn toàn)	$\mathbf{\geq 90\,\%}$	Chỉ số quan trọng nhất
Tốc độ end-to-end trên GPU	≥ 25 FPS	Video 720p
Tốc độ end-to-end trên CPU	≥ 8 FPS	Laptop không có GPU

1.3 Đóng góp

1. Một pipeline hoàn chỉnh, hỗ trợ cả biển 1 dòng và 2 dòng, từ phát hiện, tách dòng, nhận dạng, hậu xử lý đến theo dõi qua video.
2. Thuật toán hậu xử lý dựa trên mẫu định dạng biển số Việt Nam, sửa ký tự theo vị trí và chọn mẫu cần ít lần sửa nhất.
3. Bộ sinh biển số tổng hợp để huấn luyện trước CRNN, và quy trình gán nhãn chữ bán tự động (mô hình đọc nháp, người duyệt) cho biển thật.
4. Một bãi giữ xe mô phỏng 3D dùng mô hình thật: lấy thẻ, tính phí, so khớp biển ở cổng ra, xử lý mất thẻ và để bảo vệ duyệt các trường hợp bất thường; mỗi lần người gửi sửa biển số trở thành một nhãn huấn luyện.
5. Backend phân lớp (API, dịch vụ, miền, hạ tầng) với migration có phiên bản, kèm 84 bài kiểm thử tự động chạy được khi chưa có mô hình.
6. Một phân tích lỗi trung thực trên dữ liệu thật, chỉ ra vì sao độ chính xác trên dữ liệu tổng hợp không phản ánh độ chính xác thực tế.

2 Cơ sở lý thuyết và công trình liên quan

Phát hiện đối tượng. Họ mô hình YOLO [1] giải bài toán phát hiện đối tượng trong một lần lan truyền (single-stage), dự đoán đồng thời khung bao và độ tin cậy. Dự án dùng YOLO11 [2], phiên bản không dùng anchor (anchor-free) của Ultralytics. Bản *nano* chỉ có khoảng 2,6 triệu tham số, phù hợp chạy thời gian thực.

Nhận dạng chuỗi ký tự. CRNN [3] kết hợp CNN để trích đặc trưng ảnh với RNN hai chiều để mô hình hoá chuỗi, và được huấn luyện bằng hàm mất mát CTC [4]. Ưu điểm của CTC là không cần biết vị trí của từng ký tự trong ảnh, chỉ cần chuỗi đáp án của cả dòng. PaddleOCR [6] là một hệ thống OCR tổng quát mạnh, được dùng làm baseline để so sánh.

Theo dõi đa đối tượng. ByteTrack [5] theo hướng *tracking-by-detection*: nó ghép các khung phát hiện giữa các khung hình bằng bộ lọc Kalman và thuật toán ghép cặp, đồng thời tận dụng cả những khung có độ tin cậy thấp. Nhờ vậy mỗi xe giữ được một mã định danh (track ID) ổn định.

Định dạng biển số Việt Nam. Ký hiệu D là chữ số và L là chữ cái. Phần đầu biển (mã tỉnh và seri) thuộc một trong các mẫu ở Bảng 2, theo sau là 4–5 chữ số.

Bảng 2: Các mẫu đầu biển số được hỗ trợ.

Mẫu	Ví dụ	Loại xe	Cách hiển thị
`DDL`	`51G-123.45`	Ô tô	1 hoặc 2 dòng
`DDLL`	`30LD-123.45`	Ô tô (seri 2 chữ)	1 hoặc 2 dòng
`DDLD`	`59-X1 123.45`	Xe máy	2 dòng
`DDLLD`	`29-MĐ1 123.45`	Xe máy điện (chỉ seri MĐ)	2 dòng

3 Dữ liệu

3.1 Dữ liệu phát hiện biển số

Dữ liệu ảnh lấy từ bộ *vietnamese-license-plate* (phiên bản 1) trên Roboflow Universe, workspace `school-fuhih`. Nhãn ở định dạng YOLO với một lớp duy nhất là `plate`. Ba tập con có sẵn của Roboflow được gộp lại rồi chia lại theo tỉ lệ 70/15/15 với hạt giống ngẫu nhiên cố định (seed 42). Ảnh tự thu thập (tiền tố `own_`) luôn được đưa vào tập test, để tập test phản ánh điều kiện mà mô hình chưa từng thấy.

Bảng 3: Phân chia dữ liệu phát hiện.

Tập	Số ảnh	Số biển
Train	5 849	**[?]**
Validation	1 254	1 288
Test	1 254	1 280

3.2 Dữ liệu nhận dạng chữ tổng hợp

Bộ dữ liệu Roboflow chỉ có khung biển số, không có chuỗi ký tự. Để có dữ liệu huấn luyện CRNN, dự án xây dựng bộ sinh biển số tổng hợp (`src/alpr/synth.py`) gồm các bước:

1. **Sinh nội dung** ngẫu nhiên theo đúng định dạng Việt Nam: mã tỉnh 11–99, 50 % biển 2 dòng, phần số 5 chữ số với xác suất 0,8.
2. **Vẽ biển** nền sáng, chữ tối, có viền, bằng font Hershey của OpenCV.
3. **Tăng cường** để mô phỏng ảnh chụp thật: méo phối cảnh, xoay $\pm 5°$, đổi độ sáng và tương phản, làm mờ do lệch nét hoặc do chuyển động, nhiễu cảm biến, giảm độ phân giải.
4. **Tách dòng** biển 2 dòng bằng chính thuật toán dùng lúc suy luận (Mục 4.3).

Tập huấn luyện gồm 50 000 biển (75 013 ảnh dòng chữ). Tập kiểm định gồm 2 000 biển (2 976 ảnh dòng chữ) sinh với hạt giống khác. Hình 1 minh hoạ một số mẫu.



Hình 1: Biển số tổng hợp sau khi tăng cường. Font Hershey khác font biển số thật, và đây là nguồn gốc của khoảng cách miền phân tích ở Mục 8.

3.3 Nhãn chữ thật: gán nhãn bán tự động

Để fine-tune CRNN và đánh giá end-to-end, cần chuỗi ký tự của từng biển thật, lưu trong hai tệp: `texts.csv` (`image, plate_idx, text`, dùng để cắt ảnh dòng chữ huấn luyện) và `test_texts.csv` (`image, text`, đáp án đánh giá). Hai dòng của biển 2 dòng phân cách bằng dấu `/`.

Gõ tay hàng nghìn biển là không khả thi, nên dự án dùng quy trình bán tự động:

1. `scripts/draft_texts.py`: PaddleOCR đọc *nháp* mọi khung biển có trong nhãn YOLO, lưu ảnh cắt và bản nháp.
2. `scripts/review_texts.py`: giao diện duyệt từng trang ảnh cắt; người duyệt sửa chuỗi sai, đánh dấu biển không đọc được, và được báo ngay chuỗi đã gõ có đúng định dạng biển Việt Nam không.
3. Xuất hai tệp nhãn. Có thể lấy thêm *nhãn giả* (pseudo-label) cho tập train từ những bản nháp đúng định dạng và có độ tin cậy cao; tập val và test chỉ dùng nhãn đã duyệt.

Bảng 4: Tình trạng gán nhãn chữ tại thời điểm viết báo cáo.

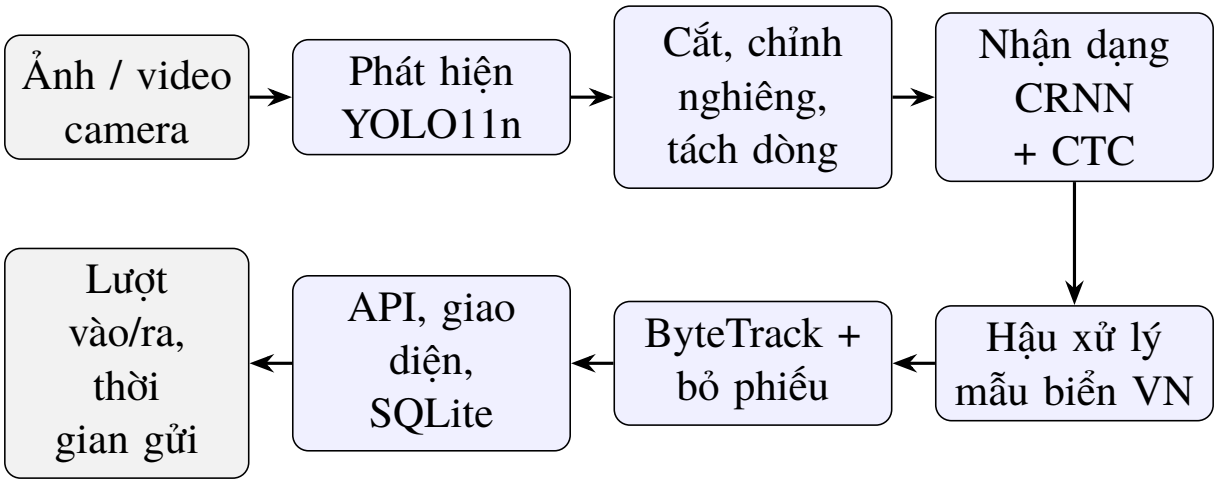
Tập	Bản nháp	Đánh dấu “đã duyệt”	Bỏ qua	Chưa duyệt
Train	2 056	169	7	1 880
Validation	311	0	0	311
Test	304	303	1	0
Tổng	2 671	472	8	2 191

Chỉ 1 502 trên 2 671 bản nháp (56 %) khớp một mẫu biển Việt Nam. Mục 8.3 cho thấy các nhãn tập test tuy đã được đánh dấu “đã duyệt” nhưng thực chất vẫn là bản nháp của PaddleOCR, nên chưa dùng làm đáp án được.

4 Phương pháp

4.1 Kiến trúc tổng thể

Hình 2 mô tả luồng xử lý. Với ảnh tĩnh, mỗi khung biển đi qua các bước từ cắt đến hậu xử lý. Với video, bộ phát hiện chạy ở chế độ theo dõi, và kết quả của từng khung hình được gộp theo track ID.



Hình 2: Kiến trúc pipeline VN-ALPR.

Mã nguồn tách thành thư viện lõi `src/alpr` (không phụ thuộc web hay CSDL), lớp ứng dụng `app/`, và các script huấn luyện, đánh giá trong `scripts/`. Bộ nhận dạng là một giao diện (protocol) chung: mọi đối tượng có phương thức `recognize(list[img]) -> list[(text, conf)]` đều dùng được. Nhờ đó CRNN và PaddleOCR thay thế được cho nhau, và kiểm thử dùng được bộ nhận dạng giả.

4.2 Phát hiện biển số

Bộ phát hiện là YOLO11n khởi tạo từ trọng số tiền huấn luyện trên COCO, rồi fine-tune với một lớp `plate`. Ngoài các phép tăng cường mặc định của Ultralytics (mosaic, HSV), cấu hình có hai điểm riêng:

- **Tắt lật ngang** (`fliplr=0`), vì biển số chứa chữ và ảnh lật gương không tồn tại ngoài thực tế.
- Xoay $\pm 10^{\circ}$ và méo phối cảnh nhẹ để mô phỏng góc đặt camera.

Lúc suy luận, khung có độ tin cậy dưới 0,4 bị loại, và NMS dùng ngưỡng IoU 0,5.

4.3 Tiền xử lý

Cắt và chỉnh nghiêng. Vùng biển được cắt với lề 5 % kích thước khung. Góc nghiêng được ước lượng từ hình chữ nhật xoay có diện tích nhỏ nhất (`minAreaRect`) bao vùng chữ đã nhị phân hoá. Góc được tính từ *cạnh dài* của bốn đỉnh hình chữ nhật, thay vì dùng giá trị góc mà OpenCV trả về, vì quy ước của giá trị đó khác nhau giữa các phiên bản OpenCV. Chỉ những góc trong khoảng $[1^{\circ}, 15^{\circ}]$ mới được hiệu chỉnh.

Phân loại 1 dòng / 2 dòng. Biển 1 dòng có tỉ lệ rộng/cao khoảng 4,7:1, biển 2 dòng khoảng 1,4:1. Ảnh cắt được coi là biển 2 dòng khi $w/h < 2{,}5$.

Tách dòng bằng hồ sơ chiếu ngang. Gọi $B(x, y) \in \{0, 1\}$ là ảnh nhị phân, trong đó điểm chữ có giá trị 1, và h là chiều cao ảnh. Hồ sơ chiếu ngang là

$$P(y) = \sum_{x} B(x, y). \tag{1}$$

Sau khi làm trơn P bằng bộ lọc trung bình 3 điểm, dòng tách y^* là tâm của đoạn các hàng có giá trị nhỏ nhất trong dải giữa $[0{,}3h,\ 0{,}7h]$, tức là hàng "trống" nhất giữa hai dòng chữ. Giới hạn trong dải giữa giúp tránh nhầm với viền trên và viền dưới của biển.

4.4 Nhận dạng ký tự: CRNN

Ảnh mỗi dòng được chuyển sang xám, co giãn giữ nguyên tỉ lệ về chiều cao 32, đệm bên phải tới chiều rộng 128, rồi chuẩn hoá về $[-1, 1]$. Kiến trúc mạng được trình bày ở Bảng 5.

Bảng 5: Kiến trúc CRNN (tổng cộng 2,35 triệu tham số). Mỗi khối Conv gồm Conv 3×3, BatchNorm và ReLU.

Lớp	Cấu hình	Kích thước đầu ra ($C \times H \times W$)
Đầu vào	ảnh xám	$1 \times 32 \times 128$
Conv + MaxPool	64, pool 2×2	$64 \times 16 \times 64$
Conv + MaxPool	128, pool 2×2	$128 \times 8 \times 32$
Conv $\times 2$ + MaxPool	256, pool 2×1	$256 \times 4 \times 32$
Conv + MaxPool	256, pool 2×1	$256 \times 2 \times 32$
Dropout2d	$p = 0{,}1$	$256 \times 2 \times 32$
Trung bình theo chiều cao		32×256 (chuỗi $T = 32$ bước)
BiLSTM	2 lớp, 128 đơn vị mỗi chiều	32×256
Tuyến tính		32×37 (36 ký tự + blank)

Hai lựa chọn thiết kế nhằm giữ mô hình xuất được sang ONNX:

- Lấy trung bình theo chiều cao bằng `mean(dim=2)` thay vì `AdaptiveAvgPool2d((1, None))`, vì phép sau không xuất được sang ONNX.
- Xuất ONNX bằng trình xuất cũ (`dynamo=False`), vì trình xuất dynamo cố định kích thước batch của LSTM vào đồ thị tính toán.

Hàm mất mát CTC. Với bộ ký tự $\mathcal{A}$ gồm 36 ký tự (0–9, A–Z) và ký hiệu trống (blank, chỉ số 0), mạng cho ra phân phối $p(\cdot \mid x)$ tại mỗi bước $t = 1, \ldots, T$. Gọi $\mathcal{B}$ là ánh xạ gộp các ký tự lặp liên tiếp rồi bỏ blank. Xác suất của chuỗi đáp án y là tổng xác suất mọi đường đi π ánh xạ về y:

$$p(y \mid x) = \sum_{\pi \in \mathcal{B}^{-1}(y)} \prod_{t=1}^{T} p(\pi_t \mid x), \qquad \mathcal{L}_{\text{CTC}} = -\log p(y \mid x). \tag{2}$$

Lúc suy luận, chuỗi được giải mã tham lam (greedy): lấy ký hiệu có xác suất lớn nhất ở mỗi bước rồi áp dụng $\mathcal{B}$. Độ tin cậy là trung bình xác suất của các ký tự được giữ lại.

4.5 Hậu xử lý theo định dạng biển số Việt Nam

Chuỗi OCR trước hết được chuẩn hoá: chuyển sang chữ hoa, đổi Đ→D, chỉ giữ A–Z và 0–9. Sau đó thuật toán thử từng mẫu đầu biển m ở Bảng 2, ghép với phần số 4–5 chữ số:

Listing 1: Thuật toán khớp mẫu biển số (rút gọn từ `plate_format.py`).

```
best = None
for m in TEMPLATES:                          # thứ tự mẫu dùng để phá thế hoà
    for n in (5, 4):                         # độ dài phần số
        if len(s) != len(m) + n: continue
        ok, text, fixes = True, "", 0
        for ch, kind in zip(s, m + "D" * n):
            if kind == "D" and not ch.isdigit():
                if ch not in TO_DIGIT: ok = False; break
                ch = TO_DIGIT[ch]; fixes += 1      # O->0, B->8, S->5, ...
            if kind == "L" and ch not in VALID_LETTERS:
                if ch not in TO_LETTER: ok = False; break
                ch = TO_LETTER[ch]; fixes += 1     # 0->D, 8->B, 5->S, ...
            text += ch
        if ok and (best is None or fixes < best.fixes):
            best = (text, fixes)
```

Ba quy tắc bổ sung:

- **Ràng buộc từ biển 2 dòng.** Độ dài dòng 1 cho biết seri kết thúc ở đâu. Tuy vậy, nếu một phép khớp trên chuỗi đã ghép cần ít lần sửa hơn, phép khớp đó được ưu tiên, để bù cho trường hợp tách dòng lệch một ký tự.
- **Thứ tự ưu tiên khi hoà.** Biển 1 dòng ưu tiên mẫu ô tô (`DDL`, `DDLL`). Biển 2 dòng ưu tiên mẫu xe máy (`DDLD`).
- **Mẫu** `DDLLD` chỉ chấp nhận seri `MD`, tức biển xe máy điện `MĐ`.

Chế độ `fix=False` chỉ chấp nhận chuỗi khớp đúng mẫu mà không sửa ký tự nào. Chế độ này dùng cho thí nghiệm ablation.

4.6 Theo dõi và bỏ phiếu qua nhiều khung hình

Trong video, mỗi xe xuất hiện ở nhiều khung hình và mỗi lần đọc có thể khác nhau. Với mỗi track ID, bộ bỏ phiếu cộng dồn điểm có trọng số cho từng chuỗi hợp lệ s:

$$S(s) = \sum_{i:\ \hat{s}_i = s} \max(c_i,\ 10^{-3}), \tag{3}$$

trong đó c_i là độ tin cậy OCR của lần đọc thứ i. Gọi $s^* = \arg\max_s S(s)$. Track được *xác nhận* và phát ra đúng một sự kiện khi thoả cả hai điều kiện sau:

$$\text{số lần đọc hợp lệ} \geq 3 \quad \text{và} \quad \frac{S(s^*)}{\sum_s S(s)} \geq 0{,}6. \tag{4}$$

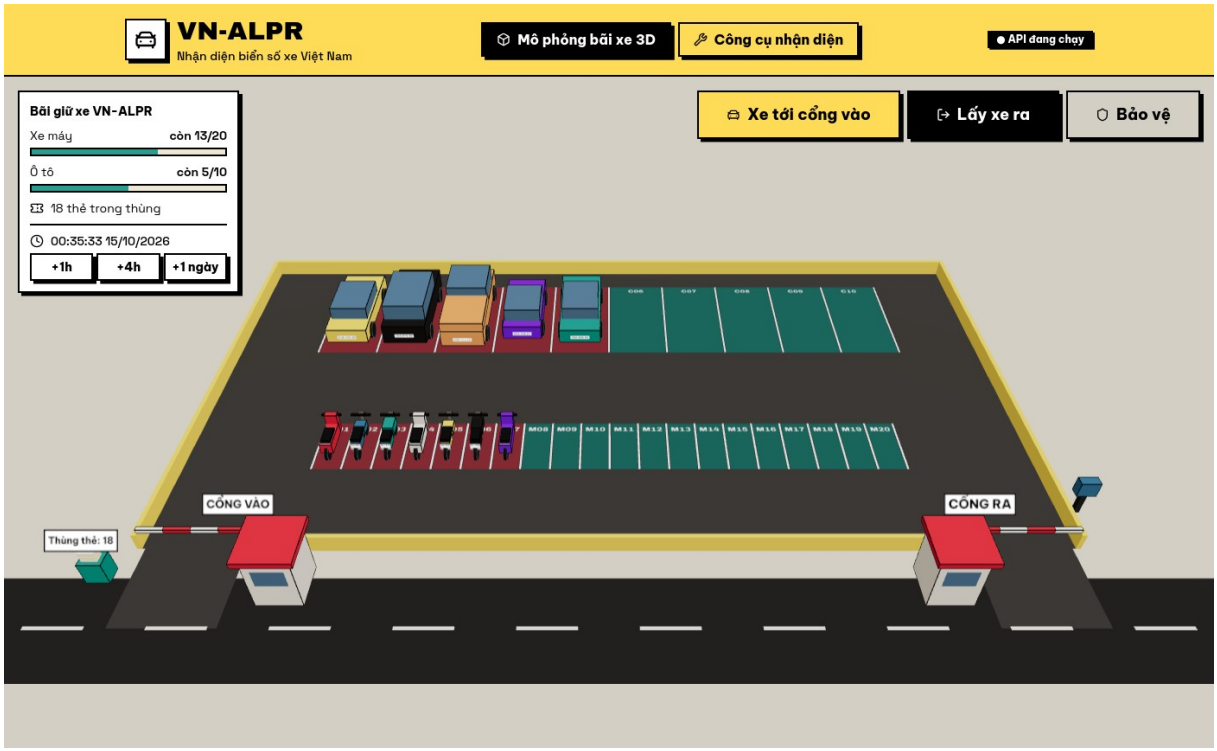
Sau khi được xác nhận, track không cần chạy OCR nữa, nhờ đó tiết kiệm tính toán. Track mất dấu quá 30 khung hình bị loại. Nếu track đó chưa được xác nhận nhưng đã có đủ số lần đọc, kết quả tốt nhất của nó vẫn được phát ra.

4.7 Nhật ký cổng

Mỗi lượt vào hoặc ra được ghi vào một nhật ký (SQLite). Một lượt cùng biển và cùng hướng trong vòng 60 giây bị bỏ qua, vì đó là cùng một xe được camera đọc hai lần. Khi xe ra mà trước đó có lượt vào, hệ thống tính thời gian gửi. Nhật ký này được dùng chung bởi API, script xử lý video `run_camera.py` và bãi giữ xe mô phỏng ở Mục 5.

5 Ứng dụng: hệ thống bãi giữ xe mô phỏng

Để đưa mô hình vào một tình huống sử dụng cụ thể, dự án xây dựng một bãi giữ xe mô phỏng 3D chạy trên trình duyệt, nối với mô hình nhận diện thật qua REST API. Người dùng đóng vai người gửi xe, bảo vệ và camera ở cổng.



Hình 3: Bãi giữ xe mô phỏng: 20 chỗ xe máy, 10 chỗ ô tô (ô đỏ là ô có xe), cổng vào với thùng thẻ ở bên trái, cổng ra với đầu đọc thẻ ở bên phải. Khung bên trái cho biết số chỗ trống, số thẻ trong thùng và đồng hồ mô phỏng.

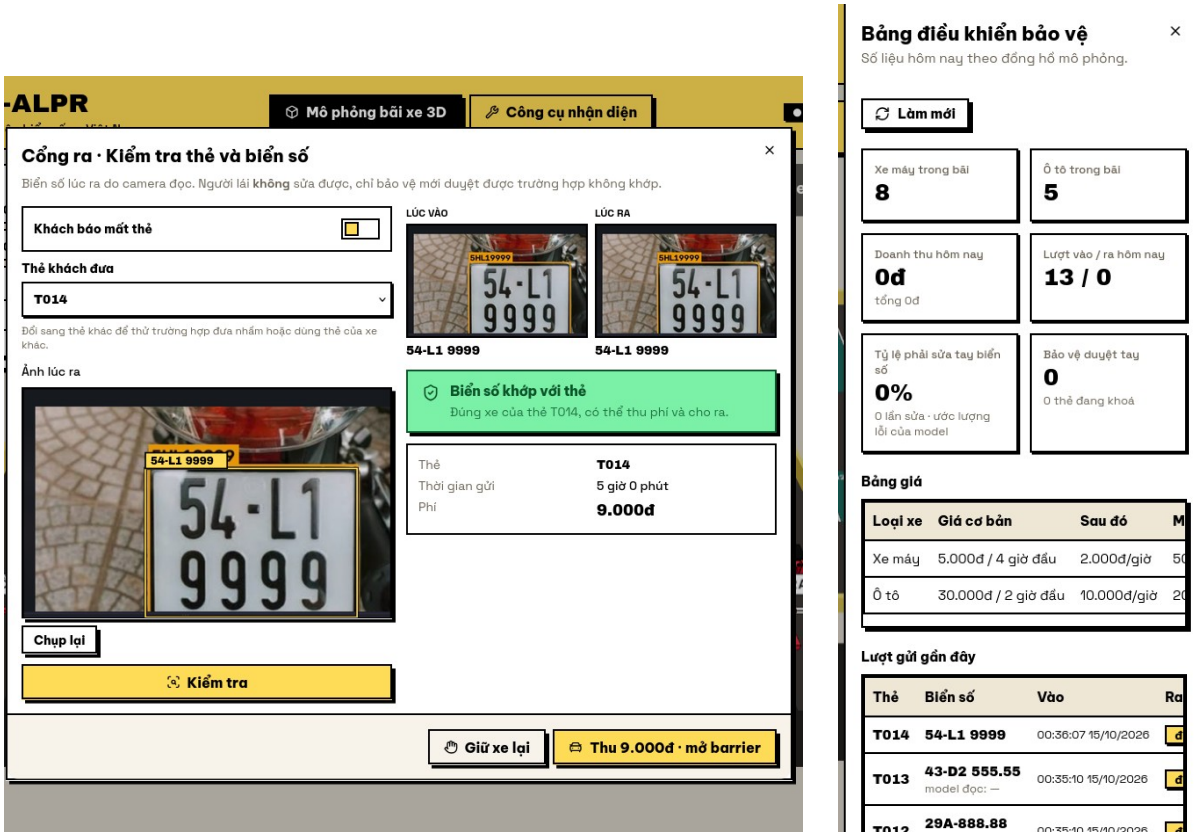
5.1 Luồng nghiệp vụ

Xe vào. Người dùng chọn loại xe, kiểu dáng và màu (4 kiểu ô tô, 3 kiểu xe máy, 9 màu), xe chạy tới cổng vào. Ảnh biển số được tải lên hoặc chụp bằng webcam. Mô hình đọc biển, và người gửi *được sửa tay* nếu mô hình đọc sai; ô nhập kiểm tra định dạng biển ngay khi gõ bằng chính hàm hậu xử lý của pipeline. Sau đó người gửi bấm vào thùng thẻ để lấy thẻ, xác nhận, barrier mở và xe chạy vào ô đỗ được cấp (Hình 4).

Xe ra. Người dùng chọn xe trong bãi, xe chạy ra cổng ra. Khách đưa thẻ (hoặc báo mất thẻ), camera chụp biển số, và hệ thống so biển đọc được với biển đã gắn với thẻ lúc vào (Hình 5a). Biển số lúc ra *không* cho người lái sửa tay: nếu được sửa, ai cầm thẻ cũng có thể gõ đúng biển để lấy xe. Khi kết quả không khớp, chỉ bảo vệ được duyệt cho xe ra và bắt buộc phải ghi lý do.



Hình 4: Cổng vào: mô hình đọc biển `54-L1 9999` với độ tin cậy 98 %, người gửi có thể sửa trước khi lấy thẻ.



(a) Cổng ra: ảnh lúc vào và lúc ra, kết quả so khớp, thời gian gửi và phí. **(b)** Bảng điều khiển bảo vệ.

Hình 5: Màn hình ở cổng ra và của bảo vệ.

5.2 Luật nghiệp vụ

- **Thẻ và chỗ đỗ.** Bãi có 30 thẻ và 30 ô (20 xe máy, 10 ô tô). Thẻ được phát theo thứ tự "trả về lâu nhất ra trước". Hệ thống từ chối khi hết chỗ cho loại xe đó, khi thẻ không còn trong thùng, hoặc khi biển số đã có trong bãi (chặn gửi trùng).
- **Phí.** Mỗi loại xe có một bảng giá: giá cơ bản cho h_0 giờ đầu, sau đó p đồng cho *mỗi giờ đã bắt đầu*:

$$\text{phí} = b + p \cdot \max(0,\ \lceil t - h_0 \rceil) + \mathbb{1}[\text{mất thẻ}] \cdot \ell, \tag{5}$$

  với t là số giờ gửi. Xe máy: $b = 5\,000$đ, $h_0 = 4$, $p = 2\,000$đ, $\ell = 50\,000$đ. Ô tô: $b = 30\,000$đ, $h_0 = 2$, $p = 10\,000$đ, $\ell = 200\,000$đ. Toàn bộ nằm trong `configs/parking.yaml`.
- **So khớp ở cổng ra.** Gọi d là khoảng cách Levenshtein giữa biển trên thẻ và biển đọc được gần nhất trong ảnh, c là độ tin cậy OCR. Kết quả là *khớp* nếu $d = 0$; *nghi đọc sai* nếu $d \le 1$ và $c < 0{,}95$ (hoặc không đọc được biển nào); còn lại là *không khớp*. Mức "nghi đọc sai" giúp bảo vệ phân biệt lỗi của mô hình với một chiếc xe khác.
- **Thẻ của xe khác.** Nếu thẻ đưa ra gắn với một xe khác trong bãi, hệ thống báo và không cho ra.
- **Mất thẻ.** Bảo vệ tra cứu theo biển số, thu thêm phí mất thẻ và *khoá* thẻ cũ để thẻ đó không mở được cổng nữa; thẻ chỉ về thùng khi bảo vệ xác nhận đã tìm thấy.

Vòng phản hồi dữ liệu. Mỗi lần người gửi sửa tay biển số ở cổng vào, hệ thống có thêm một cặp (ảnh, biển đúng). Tỉ lệ phải sửa tay được hiển thị trên bảng điều khiển như một ước lượng

độ chính xác của mô hình ngoài thực tế, và các cặp này xuất được thành tập dữ liệu dòng chữ (`labels.tsv`) để fine-tune CRNN.

Đồng hồ mô phỏng. Để trình diễn cách tính phí mà không phải chờ, đồng hồ của bãi có thể tua nhanh (+1 giờ, +4 giờ, +1 ngày). Tính năng này chỉ dùng cho demo và tắt được trong cấu hình.

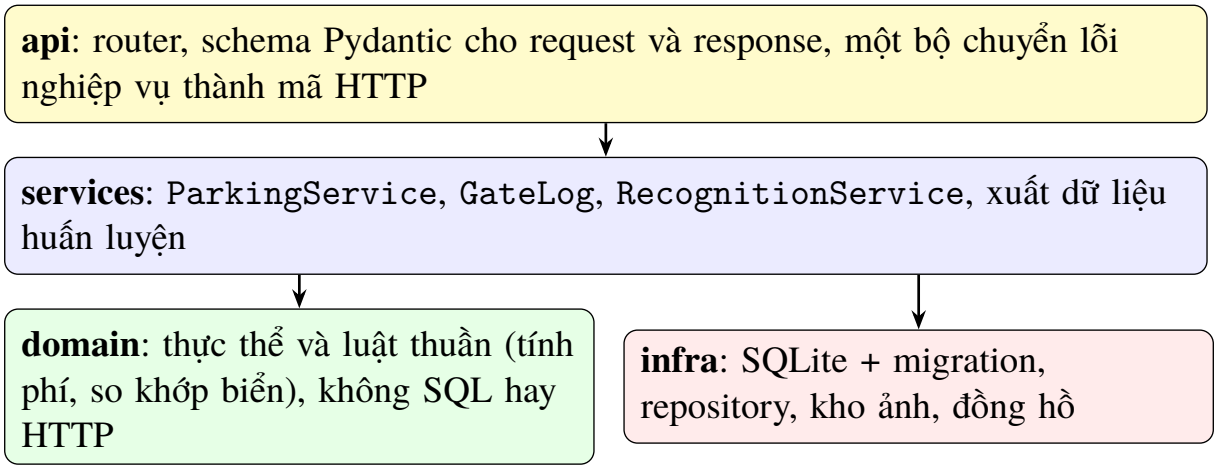
5.3 Giao diện

Giao diện viết bằng React, Vite và Tailwind CSS, theo phong cách *neobrutalism* (viền đen dày, bóng đổ cứng). Cảnh 3D dựng bằng React Three Fiber; xe là mô hình low-poly dựng từ khối hộp và trụ, biển số được vẽ lên texture từ chuỗi đã xác nhận. Một máy trạng thái duy nhất (thư viện zustand) điều khiển cả hoạt cảnh lẫn hộp thoại, để hai phần này không bao giờ lệch nhau:

Vào: chọn xe → tới cổng → nhận diện → lấy thẻ → xác nhận → vào ô đỗ
Ra: chọn xe → tới cổng → kiểm tra → ra khỏi bãi (hoặc giữ lại → về ô đỗ)

5.4 Kiến trúc backend

Backend (FastAPI) tổ chức theo kiến trúc phân lớp (Hình 6). Phụ thuộc chỉ đi một chiều: tầng API gọi tầng dịch vụ, tầng dịch vụ dùng luật của tầng miền và các giao diện mà tầng hạ tầng cài đặt. Thư viện học máy `src/alpr` tách hẳn khỏi ứng dụng.



Hình 6: Các tầng của backend. Một mô-đun "composition root" duy nhất lắp các tầng lại với nhau.

Một số quyết định đáng chú ý:

- **Giao dịch (transaction).** Ghi một lượt gửi xe, đổi trạng thái thẻ và ghi nhật ký cổng nằm trong cùng một giao dịch; khi một luật từ chối giữa chừng, không còn dữ liệu dở dang.
- **Migration có phiên bản.** Lược đồ cơ sở dữ liệu là các tệp SQL đánh số; `PRAGMA user_version` ghi lại tệp cuối đã áp dụng, nên cơ sở dữ liệu cũ tự được nâng cấp mà không mất dữ liệu.
- **Đồng hồ là một phụ thuộc.** Dịch vụ hỏi giờ qua giao diện `Clock`: thời gian thực khi chạy, đồng hồ cố định trong kiểm thử, đồng hồ tua được trong demo.

- **Kiểm thử.** 84 bài kiểm thử tự động (pytest) chạy không cần trọng số mô hình, nhờ pipeline giả và cơ sở dữ liệu tạm.

6 Thiết lập thực nghiệm

6.1 Môi trường

Mô hình được huấn luyện trên Google Colab với GPU NVIDIA T4 (15 GB) bằng notebook `notebooks/train_colab.ipynb`. Bộ phát hiện được huấn luyện hai lần với cùng cấu hình: lần 1 ngày 07/10/2026 (mất 0,52 giờ) và lần 2 ngày 14/10/2026; mô hình đang dùng là của lần 2. Tốc độ suy luận được đo trên laptop với CPU Intel Core i5-10300H (4 nhân, 8 luồng) và 14 GB RAM, không dùng GPU.

6.2 Siêu tham số

Bảng 6: Siêu tham số huấn luyện.

Bộ phát hiện (YOLO11n)	
Khởi tạo	`yolo11n.pt` (tiền huấn luyện trên COCO)
Kích thước ảnh, batch	640, 32
Số epoch	20 (cả hai lần huấn luyện)
Bộ tối ưu	AdamW, lr $= 0{,}002$, momentum 0,9 (Ultralytics tự chọn)
Tăng cường	mosaic (tắt ở 10 epoch cuối), xoay $\pm 10°$, phối cảnh $5 \cdot 10^{-4}$, HSV-V 0,5, không lật ngang
Bộ nhận dạng (CRNN)	
Dữ liệu	50 000 biển tổng hợp (75 013 dòng); kiểm định 2 000 biển (2 976 dòng). Tệp nhãn thật hiện chỉ gồm biển của tập test (Mục 3.3), nên CRNN chưa được fine-tune trên dòng chữ thật
Batch, epoch	128, 20; khoảng 115 giây/epoch trên T4. Checkpoint đang dùng là epoch 10
Bộ tối ưu	AdamW, lr $= 10^{-3}$, weight decay 10^{-4}, lịch OneCycle
Khác	cắt gradient ở chuẩn 5, CTC blank = 0, tăng cường trực tuyến với cường độ 0,7

6.3 Độ đo

- **Phát hiện:** precision, recall, mAP@0.5 và mAP@0.5:0.95 (trung bình trên các ngưỡng IoU từ 0,5 đến 0,95).
- **Độ chính xác biển** (khớp hoàn toàn): tỉ lệ biển có chuỗi dự đoán trùng hoàn toàn với đáp án sau khi chuẩn hoá.
- **Độ chính xác ký tự**: $1 - \text{CER}$, với

$$\text{CER} = \frac{\sum_i \text{lev}(\hat{y}_i, y_i)}{\sum_i |y_i|}, \tag{6}$$

trong đó lev là khoảng cách Levenshtein.

- **End-to-end:** các biển đáp án trong một ảnh được ghép tham lam với các dự đoán theo khoảng cách chỉnh sửa. Biển bị bỏ sót được tính là sai.
- **Trên ảnh cắt:** để tách riêng chất lượng nhận dạng khỏi bộ phát hiện, bộ nhận dạng còn được đánh giá trên ảnh cắt theo khung gán nhãn (cùng lề 5 % như lúc suy luận).

7 Kết quả

7.1 Bộ phát hiện

Bộ phát hiện đạt cả hai chỉ tiêu đặt ra (Bảng 7). Sau 20 epoch, mAP@0.5 đạt khoảng 0,995, tức gần như mọi biển đều được tìm thấy; mAP@0.5:0.95 vượt ngưỡng 0,70, cho thấy khung bao khá khít. Hai lần huấn luyện cho kết quả gần như trùng nhau.

Bảng 7: Kết quả bộ phát hiện YOLO11n. Tập test gồm 1 254 ảnh, 1 280 biển; mô hình đang dùng là của lần 2.

Lần huấn luyện	Tập	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Lần 1 (07/10)	test	0,991	0,991	0,995	0,713
Lần 2 (14/10)	val	0,997	0,984	0,995	0,718
Mục tiêu				$\geq 0{,}95$	$\geq 0{,}70$

7.2 Bộ nhận dạng

Trên dữ liệu tổng hợp. Trong quá trình huấn luyện, độ chính xác dòng trên tập kiểm định tổng hợp đạt 100 % từ epoch 7. Đánh giá lại CRNN đang dùng trên 1 000 biển tổng hợp *mới* (hạt giống chưa dùng khi huấn luyện) cho kết quả ở Bảng 8.

Bảng 8: CRNN trên 1 000 biển tổng hợp chưa từng thấy (1 494 dòng).

Chỉ số	Giá trị
Độ chính xác dòng	97,0 %
Độ chính xác biển, OCR thô	95,6 %
Độ chính xác biển, có hậu xử lý	95,7 %
Biển 1 dòng (506 biển)	97,0 %
Biển 2 dòng (494 biển)	94,3 %

Trên biển thật. Kết quả trên dữ liệu thật hoàn toàn khác. CRNN được đánh giá trên ảnh cắt của 154 biển tập test có nhãn là một biển số hợp lệ; số dòng được lấy theo nhãn, để loại trừ lỗi phân loại 1/2 dòng (Mục 8).

Bảng 9: CRNN trên ảnh cắt biển thật của tập test (154 biển có nhãn hợp lệ).

	Số biển	Acc. biển	Acc. ký tự
Biển 1 dòng	75	0,0 %	22,5 %
Biển 2 dòng	79	0,0 %	21,1 %
Tất cả	154	**0,0 %**	21,8 %
Mục tiêu		≥ 90 %	≥ 97 %

CRNN **không đọc đúng biển nào** và chỉ đúng khoảng một phần năm số ký tự. Ví dụ, `51A89714` được đọc thành `6L5`, `51F59011` thành `6F6UUT`. Trong khi đó, trên ảnh cận cảnh rõ nét ở Mục 8, cùng mô hình đọc đúng với độ tin cậy 98,6 %. Hai kết quả này chỉ ra khoảng cách miền giữa dữ liệu tổng hợp và dữ liệu thật, phân tích ở Mục 8.

Ablation. Bảng ablation đầy đủ (PaddleOCR so với CRNN, có và không có hậu xử lý) cần một tập đáp án đã được người duyệt. Các nhãn hiện có của tập test thực chất là bản nháp của PaddleOCR (Mục 8.3), nên mọi so sánh với PaddleOCR trên tập này đều thiên vị: PaddleOCR có hậu xử lý "đúng" 100 % chỉ vì đáp án được sinh ra từ chính nó. Vì vậy báo cáo không đưa ra con số cho PaddleOCR.

7.3 Tốc độ

Tốc độ end-to-end (phát hiện, tiền xử lý, OCR 2 dòng và hậu xử lý) được đo trên ảnh 872×541 chứa một biển 2 dòng, 30 lần sau 3 lần khởi động. Riêng phần nhận dạng (chỉnh nghiêng, tách dòng, CRNN, hậu xử lý) mất trung bình 21,9 ms cho mỗi biển.

Bảng 10: Độ trễ end-to-end trên CPU Intel i5-10300H (mô hình PyTorch).

	Trung vị (ms)	P90 (ms)	FPS (theo trung vị)
YOLO11n + CRNN	74,9	136,4	13,3
Mục tiêu (CPU)			≥ 8

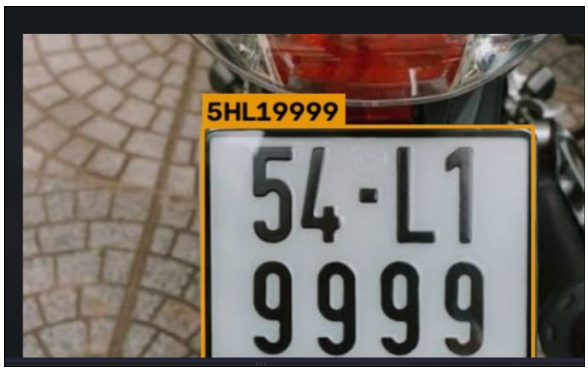
Hệ thống đạt khoảng 13 FPS trên CPU, vượt mục tiêu 8 FPS. Ở một lần đo trước với mô hình xuất sang ONNX, bản ONNX lại chậm hơn bản PyTorch (140 ms so với 62 ms), nhiều khả năng vì mô hình ONNX được xuất với đầu vào cố định 640×640, trong khi Ultralytics với `.pt` co ảnh chữ nhật về 640×416.

8 Phân tích lỗi

Bộ phát hiện hoạt động tốt; các vấn đề tập trung ở khâu nhận dạng và ở dữ liệu. Phân tích chỉ ra ba nguyên nhân độc lập.

8.1 Khoảng cách miền của bộ nhận dạng

Tình huống `54-L1 9999`**.** Hình 7 là một ảnh chụp thật. Với phiên bản CRNN đầu tiên, chuỗi đọc được là `5HL1`: số `4` bị đọc thành `H`. Lần theo pipeline cho thấy bộ phát hiện (độ tin cậy 0,85), phân loại 2 dòng và tách dòng đều đúng; lỗi nằm hoàn toàn ở CRNN. Trong font Hershey dùng để sinh dữ liệu, số `4` có đỉnh khép kín, còn trong font biển số Việt Nam, số `4` hở đỉnh và rất giống chữ `H`. Phân tích xác suất đầu ra cho thấy ở vị trí đó mô hình gần như đoán mò (`Z`, `V`, `L` cùng khoảng 0,2), nhưng nếu chỉ xét các chữ số thì `4` vẫn trội hẳn (0,13 so với 0,005). Phiên bản CRNN đang dùng đọc đúng ảnh này với độ tin cậy 98,6 %.



(a) Ảnh chụp thật (khung do bộ phát hiện vẽ).



(b) Hai ảnh dòng chữ CRNN nhận được (32×128, phóng to).

Hình 7: Tình huống `54-L1 9999`: phiên bản CRNN đầu đọc thành `5HL1`, phiên bản hiện tại đọc đúng.

Trên bộ dữ liệu. Tuy đọc đúng ảnh cận cảnh rõ nét, CRNN không đọc đúng biển nào trong 154 biển của tập test (Bảng 9), trong khi đạt 95,7 % trên biển tổng hợp. Ảnh cắt trong bộ dữ liệu khác dữ liệu tổng hợp ở nhiều mặt cùng lúc: độ phân giải thấp, font biển thật, chữ bị co hẹp do ảnh bị co giãn (Mục 8.2), cùng bóng đổ, ốc vít và khung biển. Mô hình chỉ học trên dữ liệu tổng hợp nên học thuộc hình dạng của font Hershey thay vì học đặc trưng chung của chữ số. Đây là **điểm nghẽn chính** của hệ thống.

8.2 Phân loại 1 dòng / 2 dòng bằng tỉ lệ khung

Pipeline coi một ảnh cắt là biển 2 dòng khi tỉ lệ rộng/cao nhỏ hơn 2,5, dựa trên giả định biển ô tô 1 dòng có tỉ lệ khoảng 4,7:1. Trên tập test, giả định này không còn đúng:

Bảng 11: Tỉ lệ rộng/cao của ảnh cắt biển trong tập test và kết quả phân loại với ngưỡng 2,5.

Loại biển (theo nhãn)	Số biển	Trung vị rộng/cao	Bị coi là 2 dòng
1 dòng	98	2,08	70 (71 %)
2 dòng	205	1,08	205 (100 %)

Biển 1 dòng chỉ còn tỉ lệ khoảng 2:1, thấp hơn nhiều so với kích thước thật. Nhiều khả năng ảnh trong bộ dữ liệu đã bị co giãn về 640×640 khi xuất từ Roboflow, làm biển bị bóp hẹp theo chiều ngang. Hệ quả là 71 % biển 1 dòng bị cắt đôi trước khi đưa vào OCR, làm hỏng kết quả của cả CRNN lẫn bản nháp của PaddleOCR. Đặt ngưỡng cố định theo kích thước biển thật là không đủ an toàn; cách phân loại cần dựa vào chính nội dung ảnh (Mục 9).

8.3 Chất lượng nhãn chữ

Toàn bộ 303 nhãn được đánh dấu "đã duyệt" của tập test **trùng khớp hoàn toàn** với bản nháp của PaddleOCR, và chỉ 154 nhãn trong số đó là biển số hợp lệ. Nhìn vào ảnh cắt có thể thấy nhãn sai rõ ràng (Hình 8): biển `51A-775.29` được gán `00/51A77529`, biển `51A-052.27` được gán `51A05227/JTAUSEILA`. Những nhãn này đến từ chính lỗi tách dòng ở Mục 8.2: PaddleOCR đọc từng nửa biển và trả về cả các ký tự rác.



Hình 8: Ảnh cắt biển trong tập test và nhãn đang có (chữ đỏ). Nhãn là bản nháp của PaddleOCR, chưa được sửa.

Bài học rút ra là bước duyệt nhãn phải thật sự là người kiểm tra từng ảnh, đặc biệt với tập test: một tập đáp án sinh ra từ một mô hình sẽ làm mọi phép so sánh thiên vị về phía mô hình đó.

8.4 Nhận xét chung

- Bộ phát hiện đạt chỉ tiêu và không phải là điểm nghẽn.
- Độ chính xác trên dữ liệu tổng hợp *không* phản ánh độ chính xác thật; đánh giá trên ảnh thật với nhãn đã duyệt là bắt buộc.
- Ba vấn đề cộng dồn với nhau: lỗi tách dòng làm hỏng bản nháp, nhãn hỏng chưa được sửa, và CRNN chưa từng thấy biển thật. Sửa theo đúng thứ tự (phân loại dòng, duyệt nhãn, rồi fine-tune) mới tránh được việc huấn luyện trên dữ liệu sai.

9 Hạn chế và hướng phát triển

Hạn chế.

1. CRNN mới chỉ học trên dữ liệu tổng hợp và chưa đọc được biển trong bộ dữ liệu thật; chỉ tiêu độ chính xác biển ($\geq 90\,\%$) chưa đạt.
2. Nhãn chữ của tập test chưa được người duyệt, nên chưa có số liệu ablation end-to-end đáng tin cậy.
3. Phân loại 1 dòng / 2 dòng bằng ngưỡng tỉ lệ khung cố định thất bại khi ảnh bị co giãn.
4. Chưa tối ưu cho ảnh ban đêm hay camera hồng ngoại; hướng vào/ra cố định theo camera; giao diện web chưa có trang xử lý video.

Hướng phát triển, theo thứ tự ưu tiên.

1. **Phân loại số dòng dựa trên nội dung ảnh.** Thay ngưỡng tỉ lệ khung bằng việc tìm hai vùng chữ tách rời trong hồ sơ chiếu ngang (chính hồ sơ đang dùng để tách dòng), hoặc một bộ phân loại nhỏ. Sau đó chạy lại bước tạo nhãn nháp.
2. **Duyệt nhãn thật sự**, bắt đầu từ tập test, rồi tập val. Với tập train có thể kết hợp nhãn đã duyệt và nhãn giả có độ tin cậy cao.
3. **Fine-tune CRNN** trên ảnh dòng chữ thật, sau đó chạy ablation end-to-end (YOLO + PaddleOCR / CRNN, có và không có hậu xử lý) bằng `scripts/evaluate.py`.
4. **Giải mã CTC có ràng buộc theo mẫu biển.** Ở vị trí bắt buộc là chữ số chỉ chọn trong các chữ số, ở vị trí bắt buộc là chữ cái chỉ chọn trong các chữ cái hợp lệ, và chọn mẫu có xác suất cao nhất. Cách này dùng xác suất của chính mô hình thay cho bảng ký tự dễ nhầm viết tay; với tình huống `54-L1`, chữ số có xác suất cao nhất ở vị trí bị đọc sai chính là `4`.
5. Sinh dữ liệu tổng hợp bằng font gần với font biển số Việt Nam, kèm co giãn ảnh, để giảm khoảng cách miền ngay từ giai đoạn huấn luyện trước.
6. Trong lúc CRNN chưa đạt yêu cầu, ứng dụng có thể chuyển sang PaddleOCR bằng một dòng cấu hình (`ocr.engine: paddle`).
7. Xuất ONNX với kích thước đầu vào động; suy ra hướng vào/ra từ quỹ đạo theo dõi; thêm trang xử lý video vào giao diện web.

10 Kết luận

Dự án đã xây dựng một hệ thống ALPR hoàn chỉnh cho biển số Việt Nam, xử lý được cả biển 1 dòng và 2 dòng, đi từ phát hiện, nhận dạng, hậu xử lý theo định dạng biển, theo dõi qua video đến một ứng dụng bãi giữ xe mô phỏng 3D có lấy thẻ, tính phí, so khớp biển ở cổng ra và bảo vệ duyệt các trường hợp bất thường. Backend được tổ chức theo kiến trúc phân lớp với 84 bài kiểm thử tự động.

Bộ phát hiện YOLO11n đạt cả hai chỉ tiêu (mAP@0.5 = 0,995, mAP@0.5:0.95 = 0,713 trên tập test), và hệ thống chạy khoảng 13 FPS trên CPU laptop. Ngược lại, CRNN huấn luyện trên dữ liệu tổng hợp đạt 95,7 % trên biển tổng hợp nhưng 0 % trên biển thật của bộ dữ liệu. Phân tích lỗi chỉ ra ba nguyên nhân cộng dồn: khoảng cách miền của dữ liệu tổng hợp, phân loại số dòng bằng tỉ lệ khung thất bại trên ảnh bị co giãn, và nhãn chữ chưa được duyệt thật. Các bước tiếp theo theo đúng thứ tự là sửa cách phân loại số dòng, duyệt nhãn, fine-tune CRNN trên biển thật và hoàn thiện đánh giá end-to-end.

Tài liệu

[1] J. Redmon, S. Divvala, R. Girshick, A. Farhadi. You Only Look Once: Unified, Real-Time Object Detection. In *CVPR*, 2016.

[2] G. Jocher, J. Qiu. Ultralytics YOLO11. `https://github.com/ultralytics/ultralytics`, 2024.

[3] B. Shi, X. Bai, C. Yao. An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition. *IEEE TPAMI*, 39(11):2298–2304, 2017.

[4] A. Graves, S. Fernández, F. Gomez, J. Schmidhuber. Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. In *ICML*, 2006.

[5] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, X. Wang. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. In *ECCV*, 2022.

[6] Y. Du, C. Li, R. Guo, X. Yin, W. Liu, J. Zhou, Y. Bai, Z. Yu, Y. Yang, Q. Dang, H. Wang. PP-OCR: A Practical Ultra Lightweight OCR System. *arXiv:2009.09941*, 2020.

A Cấu trúc mã nguồn

```
src/alpr/          thư viện lõi
  detection/       YOLO: detect() cho ảnh, track() cho video (ByteTrack)
  preprocess.py    cắt, chỉnh nghiêng, phân loại và tách dòng
  ocr/crnn/        mô hình, bộ ký tự, dataset, bộ nhận dạng .pt/.onnx
  ocr/paddle.py    baseline PaddleOCR
  postprocess/     khớp mẫu biển số Việt Nam
  tracking/        bỏ phiếu theo track ID
  pipeline.py      ghép các khối
app/
  api/             router, schema; main.py lắp ráp ứng dụng
  services/        bãi xe, nhật ký cổng, nhận diện, xuất dữ liệu
  domain/          thực thể, tính phí, so khớp biển
  infra/           SQLite + migration, repository, ảnh, đồng hồ
web/               giao diện React, mô phỏng bãi xe 3D
scripts/           dữ liệu, gán nhãn, huấn luyện, đánh giá, xuất, benchmark
tests/             pytest -- dùng mô hình giả nên không cần trọng số
```

B Tái lập kết quả

```
# huấn luyện: notebooks/train_colab.ipynb trên Colab (GPU T4)
pip install -r requirements.txt && pip install -e .
pytest -q                                          # kiểm thử
cd web && npm install && npm run dev               # giao diện web
uvicorn app.api.main:app --port 8000               # REST API
python scripts/evaluate.py --images data/processed/detection/images/test \
    --gt data/raw/test_texts.csv --tag "YOLO + CRNN + postprocess"
```